OBJECT ORIENTED PROGRAMMING LANGUAGE

Semester : “BSCS 2ND”

Section: “C ”

Submitted To:

DR SAMIA IJAZ

&

SIR. MUHAMMAD HUSNAIN

Submitted By:

EHRAM

22-CS-027

HITEC UNIVERSITY TAXILA

TRAFFIC CONTROL SYSTEM

INTRODUCTION

The Traffic Control Center System is a software solution developed to effectively manage and monitor different types of vehicles on a road network. This system aims to provide comprehensive functionalities such as vehicle registration, traffic speed monitoring, and violation detection. By modeling the traffic control operations, the system enables efficient management and enforcement of traffic regulations.

The system is designed using object-oriented programming principles, where a base class called "Vehicle" serves as the foundation for modeling common attributes shared by all vehicles. The Vehicle class includes essential properties like registration number, brand, and maximum speed, along with appropriate getter and setter methods for accessing and modifying these attributes.

To accommodate different types of vehicles, the system implements derived classes, such as "Car," "Motorcycle," and "Truck." Each derived class inherits from the base class, Vehicle, and extends its functionality by including additional attributes specific to the corresponding vehicle type. This inheritance hierarchy allows for code reuse and provides a structured representation of the various vehicles.

The Vehicle class also overloads the comparison operator (==) to enable comparing vehicles based on their registration numbers. This facilitates the efficient identification and retrieval of vehicles when needed.

The core functionality of the Traffic Control Center System is implemented through the TrafficControlCenter class. This class serves as the central management component and handles operations related to vehicle registration, traffic speed monitoring, and violation detection. By utilizing polymorphism, the TrafficControlCenter class can treat different types of vehicles uniformly during speed monitoring and violation detection processes. This polymorphic behavior simplifies the handling of vehicles and allows for seamless integration of new vehicle types in the future.

METHODOLOGIES

Object-Oriented Programming (OOP): The program utilizes OOP principles to define classes and objects. Each vehicle type (Car, Motorcycle, Truck) is implemented as a class that inherits from the base class Vehicle. This allows for code reuse, encapsulation, and polymorphism.

Encapsulation: The member variables of the classes (registrationNumber, brand, maximumSpeed, color, owner, price) are declared as private, encapsulating the data within the class. Access to these variables is provided through public getter methods, such as getRegistrationNumber(), getBrand(), and getMaximumSpeed(). This ensures data integrity and provides a controlled interface for accessing the class data.

Polymorphism: The program demonstrates polymorphism by defining a pure virtual function show() in the base class Vehicle. The derived classes (Car, Motorcycle, Truck) override this function to provide their own implementation. The show() function is then called on objects of different derived classes, which is resolved at runtime based on the actual object type.

Dynamic Memory Allocation: The TrafficControlCenter class maintains a vector of pointers to Vehicle objects, allowing for dynamic memory allocation of objects based on user input during vehicle registration. Memory is allocated using the **new** operator, and deallocated using the **delete** operator in the destructor of TrafficControlCenter.

Input/Output: The program uses the standard input/output streams (cin and cout) to interact with the user. It prompts the user for various inputs such as vehicle type, registration number, brand, maximum speed, color, owner, price, and speed limit. The program then displays relevant information and messages to the user.

Control Flow: The program uses control flow constructs such as if-else statements and switch-case statements to handle user choices and perform different operations based on the selected options. It uses a do-while loop to repeatedly display the menu and accept user input until the user chooses to exit the program.

Exception Handling: The program does not explicitly handle exceptions. However, it relies on the correct input format from the user. If the user enters invalid input or exceeds the capacity of the input variables, the behavior may be undefined.

Code Organization: The program is organized into multiple classes, each serving a specific purpose. This helps in modularizing the code and separating concerns. The main() function serves as the entry point of the program and orchestrates the interactions with the TrafficControlCenter object.

THE CODE:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

class Vehicle {
private:
    int registrationNumber;
    string brand;
    int maximumSpeed;

public:
    Vehicle(int regNo, string brnd, int speed)
        : registrationNumber(regNo), brand(brnd), maximumSpeed(speed) {}

    int getRegistrationNumber() {
        return registrationNumber;
    }

    string getBrand() {
        return brand;
    }

    int getMaximumSpeed() {
        return maximumSpeed;
    }
```

```
    virtual void show() = 0;

    bool operator==(const Vehicle& other) {
        return registrationNumber == other.registrationNumber;
    }
};

class Car : public Vehicle {
private:
    string color;

public:
    Car(int regNo, string brnd, int speed, string clr)
        : Vehicle(regNo, brnd, speed), color(clr) {}

    void show() {
        cout << "Car with registration number " << getRegistrationNumber()
            << ", brand " << getBrand()
            << ", maximum speed " << getMaximumSpeed()
            << ", and color " << color << endl;
    }
};

class Motorcycle : public Vehicle {
private:
    string owner;

public:
    Motorcycle(int regNo, string brnd, int speed, string ownr)
```

```
        : Vehicle(regNo, brnd, speed), owner(ownr) {}

    void show() {
        cout << "Motorcycle with registration number " << getRegistrationNumber()
            << ", brand " << getBrand()
            << ", maximum speed " << getMaximumSpeed()
            << ", and owned by " << owner << endl;
    }
};

class Truck : public Vehicle {
private:
    string np;

public:
    Truck(int regNo, string brnd, int speed, string np)
        : Vehicle(regNo, brnd, speed), np(np) {}

    void show() {
        cout << "Truck with registration number " << getRegistrationNumber()
            << ", brand " << getBrand()
            << ", maximum speed " << getMaximumSpeed()
            << ", and number plate " << np << endl;
    }
};

class TrafficControlCenter {
private:
    vector<Vehicle*> vehicles;
```

```
public:
    ~TrafficControlCenter() {
        for (vector<Vehicle*>::iterator it = vehicles.begin(); it != vehicles.end(); ++it) {
            delete *it;
        }
        vehicles.clear();
    }

    void registerNewVehicle() {
        int choice;
        cout << "Select the type of vehicle to register:" << endl;
        cout << "1. Car" << endl;
        cout << "2. Motorcycle" << endl;
        cout << "3. Truck" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        int regNo, speed;
        string brand;
        string additionalAttribute;

        cout << "Enter registration number: ";
        cin >> regNo;
        cout << "Enter brand: ";
        cin.ignore();
        getline(cin, brand);
        cout << "Enter speed: ";
        cin >> speed;

        Vehicle* vehicle = NULL;
```

```
switch (choice) {
    case 1: {
        string color;
        cout << "Enter color: ";
        cin.ignore();
        getline(cin, color);
        vehicle = new Car(regNo, brand, speed, color);
        break;
    }
    case 2: {
        string owner;
        cout << "Enter owner: ";
        cin.ignore();
        getline(cin, owner);
        vehicle = new Motorcycle(regNo, brand, speed, owner);
        break;
    }
    case 3: {
        string np;
        cout << "Enter plate number: ";
        cin >> np;
        vehicle = new Truck(regNo, brand, speed, np);
        break;
    }
    default:
        cout << "Invalid choice. Vehicle registration failed." << endl;
        return;
}

vehicles.push_back(vehicle);
```

```
        cout << "Vehicle registered successfully." << endl;
    }

    void monitorTrafficSpeed() {
        cout << "Current registered vehicles:" << endl;
        for (vector<Vehicle*>::iterator it = vehicles.begin(); it != vehicles.end(); ++it) {
            (*it)->show();
        }
    }

    void detectViolation(int speedLimit) {
        cout << "Violations detected (speed limit: " << speedLimit << "):" << endl;
        bool violationDetected = false;
        for (vector<Vehicle*>::iterator it = vehicles.begin(); it != vehicles.end(); ++it) {
            if ((*it)->getMaximumSpeed() > speedLimit) {
                (*it)->show();
                violationDetected = true;
            }
        }
        if (!violationDetected) {
            cout << "No vehicles detected exceeding the speed limit." << endl;
        }
    }
};

int main() {
    TrafficControlCenter tcc;
    int choice;

    do {
```

```
    cout << "\nSelect an option:" << endl;
    cout << "1. Register a new vehicle" << endl;
    cout << "2. Monitor traffic speed" << endl;
    cout << "3. Detect speed violation" << endl;
    cout << "4. Exit" << endl;
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
        case 1:
            tcc.registerNewVehicle();
            break;
        case 2:
            tcc.monitorTrafficSpeed();
            break;
        case 3: {
            int speedLimit;
            cout << "Enter the speed limit: ";
            cin >> speedLimit;
            tcc.detectViolation(speedLimit);
            break;
        }
        case 4:
            cout << "Exiting program..." << endl;
            break;
        default:
            cout << "Invalid choice. Please try again." << endl;
    }
} while (choice != 4);
```

```
    return 0;
}
```

OUTPUTS OF THE CODE:

```
C:\Users\ayesh\Desktop\Programs\ccp.exe

Select an option:
1. Register a new vehicle
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 1
Select the type of vehicle to register:
1. Car
2. Motorcycle
3. Truck
Enter your choice: 1
Enter registration number: 01
Enter brand: Audi
Enter speed: 90
Enter color: black
Vehicle registered successfully.

Select an option:
1. Register a new vehicle
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 1
Select the type of vehicle to register:
1. Car
2. Motorcycle
3. Truck
Enter your choice: 2
```

C:\Users\ayesh\Desktop\Programs\ccp.exe

```
Enter your choice: 2
Enter registration number: 02
Enter brand: honda
Enter speed: 85
Enter owner: Mustafa
Vehicle registered successfully.

Select an option:
1. Register a new vehicle
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 1
Select the type of vehicle to register:
1. Car
2. Motorcycle
3. Truck
Enter your choice: 3
Enter registration number: 03
Enter brand: Hino
Enter speed: 76
Enter plate number: ESQ-334
Vehicle registered successfully.

Select an option:
1. Register a new vehicle
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 2
```

```
C:\Users\ayesh\Desktop\Programs\ccp.exe
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 2
Current registered vehicles:
Car with registration number 1, brand Audi, maximum speed 90, and color black
Motorcycle with registration number 2, brand honda, maximum speed 85, and owned by Mustafa
Truck with registration number 3, brand Hino, maximum speed 76, and number plate ESQ-334

Select an option:
1. Register a new vehicle
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 3
Enter the speed limit: 85
Violations detected (speed limit: 85):
Car with registration number 1, brand Audi, maximum speed 90, and color black

Select an option:
1. Register a new vehicle
2. Monitor traffic speed
3. Detect speed violation
4. Exit
Enter your choice: 4
Exiting program...

--------------------------------
Process exited after 624.7 seconds with return value 0
Press any key to continue . . .
```

EXPLAINATION

The above code is an implementation of a traffic control center program. It demonstrates a simple traffic control center system where vehicles can be registered, their details can be monitored, and speed violations can be detected. The code includes necessary libraries for input/output operations <iostream>, file operations <fstream>, string manipulation <string > , data container < vector>, and algorithms < algorithm>. The **using namespace std** statement allows us to use names from the **std** namespace without explicitly qualifying them. The code defines a base class called **Vehicle** that represents a generic vehicle. It has private data members for registration number, brand, and maximum speed. It provides a constructor to initialize these members, as well as getter functions to retrieve the values. The class also declares a pure virtual function **show()** which makes Vehicle an abstract class that cannot be instantiated directly. The **bool operator==** overload allows comparing two vehicles based on their registration numbers. The code also includes three derived classes: **Car, Motorcycle,** and **Truck.** These classes inherit from the Vehicle class and add additional features specific to each type of vehicle. Each derived class has its own constructor that initializes the base class members using the initialization list and sets the additional features. The derived classes also override the **show()** function from the base class to display information specific to each type of vehicle. The code further defines a **TrafficControlCenter** class that manages the

registration and monitoring of vehicles. It has a private member vehicle which is a vector of pointers to Vehicle objects.

The **TrafficControlCenter** class provides the following member functions:

- **registerNewVehicle():** This function allows the user to register a new vehicle by selecting the type of vehicle (car, motorcycle, or truck) and entering the required information such as registration number, brand, speed, and additional attributes specific to each type of vehicle. Based on the user's choice, an appropriate derived class object is created dynamically using **new** operator and added to the **vehicles** vector.
- **monitorTrafficSpeed():** This function displays the details of all the registered vehicles by calling the **show()** function for each vehicle object in the vehiclevector.
- **detectViolation(int speedLimit):** This function detects and displays any vehicles that exceed the specified speed limit. It iterates over the **vehicles** vector, checks the maximum speed of each vehicle, and if it exceeds the speed limit, calls the **show()** function to display its details.
- The **main()** function creates an instance of the **TrafficControlCenter** class and presents a menu to the user. It allows the user to choose options such as registering a new vehicle, monitoring traffic speed, detecting speed violations, or exiting the program. The chosen option is executed until the user selects the exit option.

CONCLUSION

In conclusion, we have successfully developed a traffic control system that models the management of different types of vehicles on a road network. The system fulfills the requirements by implementing a base class called "Vehicle" to store common attributes and derived classes for specific vehicle types such as "Car," "Motorcycle," and "Truck." Polymorphism is used effectively in the "TrafficControlCenter" class to handle different vehicle types uniformly when monitoring traffic speed and detecting violations. The system allows for the registration of new vehicles, storing their information such as registration number, brand, and maximum speed. It provides functions to monitor the traffic speed by displaying the details of all registered vehicles, and to detect violations by identifying vehicles that exceed a given speed limit. The use of object-oriented programming principles, such as inheritance and operator overloading, enhances the system's flexibility and reusability. The code is well-structured and modular, facilitating future enhancements and maintenance.

Overall, this traffic control system provides a solid foundation for managing and monitoring vehicles on a road network. It can be further extended to include additional features such as generating reports, implementing traffic rules, or integrating with external systems. With ongoing development and refinement, this system has the potential to contribute to efficient and safe traffic management in real-world scenarios.